

1 Short questions: 3 marks each

1. Let a process **P** create a child process **P1** and a process **P'** create a child process **P2**. Assuming **P** and **P'** have died while both **P1** and **P2** are still alive, what will be the **ppids** of **P1** and **P2**
Your answer : 1 (init, whose ppid is 1, adopts all orphans).
2. It is possible for a process to ignore all incoming signals.
Your answer(True/False) : NO (Except for SIGKILL and SIGSTP).
3. When the system call **exit()** fails, it returns -1?
Your answer(True/False) : False (exit() never returns and never fails).
4. A process has the option to change its own PPID.
Your answer (True/False) : False (this is done by the kernel, not the process).
5. What happens to a process that receives the SIGKILL signal from the user root?
Your answer : Terminated (SIGKILL is fatal).
6. A shell script must be compiled before it becomes executable.
Your answer(True/False) : False
7. When a process is killed, all its children die as a consequence?
Your answer(True/False) : False
8. The system call **read(0, buff, 100)** reads a 100 bytes from the keyboard into array **buff**.
Your answer(True/False) : False, reads up to 100.
9. A process with children will not be blocked by the system call **wait()**.
Your answer(True/False) : False
10. What will be printed by the statement below, assuming **fd** is a valid file descriptor:

```
printf("%d\n", lseek(fd, 000L, SEEK_SET)) ;
```

Your answer : 0

2 Tracing: 7 marks each

1. How many "Hi"s will this program print?

```
main() {  
    for(int i=0; i < 5; i++)  
        fork();  
  
    printf("Hi\n");  
}
```

Your answer : $2^5=32$ (the for loop creates 2 to the power of 5, that is 32, all will print)

2. How many "Hi"s will this program print?

```
main() {
    fork();
    for(int i=0; i < 3; i++){
        if(!fork())
            printf("Hi\n");
    }
}
```

Your answer : $2+4+8=14$

3. What will appear on the screen when the following program runs?

```
int main(int argc, char *argv[]){
    printf("Hello\n");
    fork();
    printf("Windsor");
    pause();
    printf("Bye\n");
    exit(0);
}
```

Your answer : Hello (because of buffering and pause())

4. In a few words, what does the bash script below, **mySc**, when we type **mySc 10000**.

```
for name in `ls *.exe`
do
    if [[ `cat $name | wc -c` > $1 ]]; then
        rm $name
    fi
done
```

Your answer : Delete all .exe files with sizes greater than 10000

5. What will be printed by the program below?

```
int main(void) {
    int pid;

    pid=fork();
    alarm(3);
    if(!pid){
        sleep(2);
        printf("Saturday\n");
    }else
        printf("Today\n");

    sleep(1);
    printf("Tomorrow\n");
    exit(0);
}
```

Your answer : Today\n Tomorrow\n Saturday (child will die before printing Tomorrow).

6. What will appear on the screen:

```
main() {
    int n, status;

    if(fork())
        n=100;
    else
        n=200;

    wait(&status);
    printf("%d\n", n);
    exit(0);
}
```

Your answer :

200

100 because wait() blocks the parent, so child will always print first.

